

Lab 3: AWS Security and Server Infrastructure

Aim

The aim of this lab is to set up AWS server instances and understand the setup of key security aspects. It is preferable to use a **Ubuntu server** rather than an Amazon Linux instance.

Activities

A demo of the setup of the lab is here

Outline

In the previous lab, we have set up a range of architectures with GN3. This is a private cloud environment and creates infrastructure-as-a-service. Increasingly, we use the public cloud to build our information systems, which reduces data centre investment costs while providing the opportunity to quickly scale our server, network, and data infrastructure. It is generally a pay-as-you-go model, and we pay for CPU time, network bandwidth and data costs. The most popular public cloud provider is AWS (Amazon Web Services), and which provides EC2 (for compute), S3 (for data buckets), RDS (for databases) and AWS Network Firewall (for firewalls). Some of these services are outlined in Figure 1.

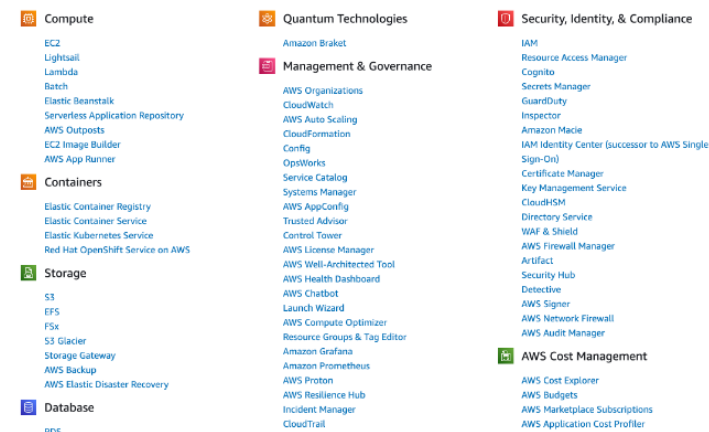


Figure 1: AWS Services

Enabling your lab

You should have an AWS Academy login, so go to: <https://awsacademy.instructure.com/> and log into the system and select AWS Academy Learner Lab (Figure 2).

AWS Academy Learner Lab - Foundation Services [28224]



This is the Edinburgh Napier University AWS Academy Learner Lab for C5N09112, and which has \$100 of credit for you to learn AWS Services. If you exceed your credit, you will lose your environment, so remember to switch your instances off when you are not using them. The lab session is four hours, but you can restart the lab again at any time. All EC2 instances will be shut off after the four hours are up. Do not end the lab, until we have finished with the module. Some resources are:

- Introduction to Cloud Computing: [here](#) ↗.
- A demo of setting up EC2 instances is [here](#) ↗.

Click [Modules](#) to start the course.

Figure 2: AWS Academy Learner Lab

Next, select “Modules”, and then “Learner Lab - Foundational Services”, and you should have the lab environment (Figure 3).

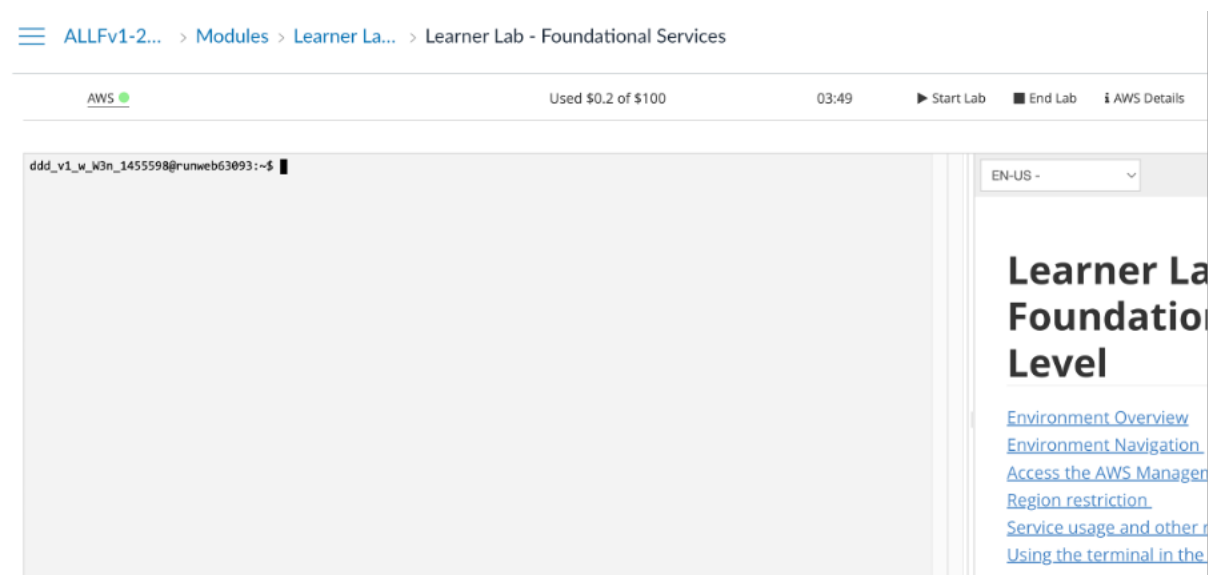


Figure 3: AWS Academy Learner Lab environment

In the console, you can interact with your AWS through the console (as you are already logged into AWS). Now, press the “Start Lab” button, and wait for the AWS light to go green. Once it turns green, you can click on it and open up your AWS Management console. After this, just select EC2, and you should see your EC2 environment.

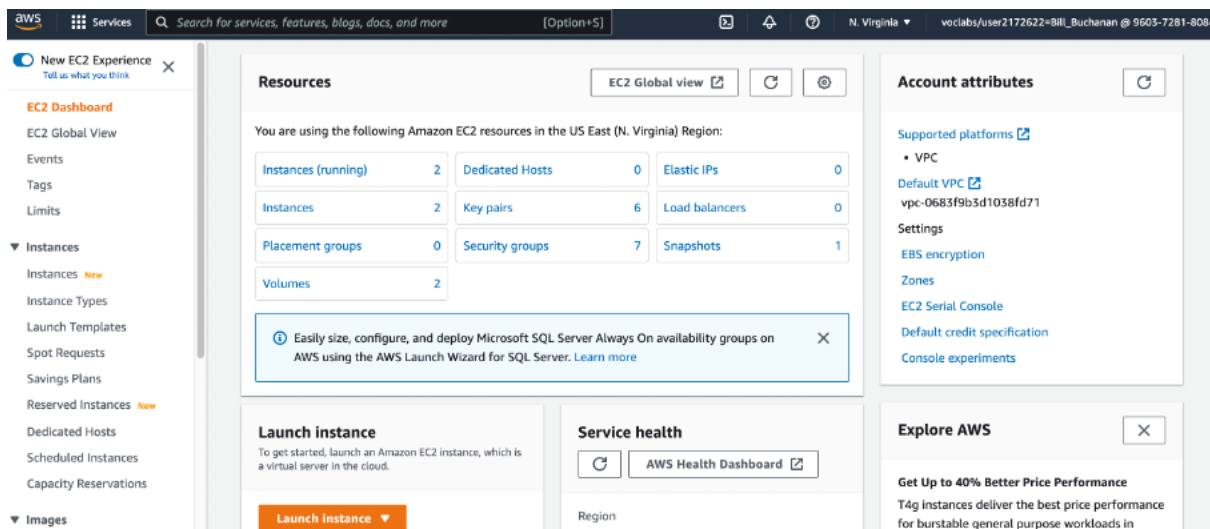


Figure 4: AWS Management Console (EC2)

Creating and Securing a Ubuntu Server

We will now create a Ubuntu Server, and which should be accessible from the Internet. For this, select “Launch Instance”, and then give it a name (such as “My Linux Server”) and select the Ubuntu instance for the AMI (Amazon Machine Image) – as shown in Figure 5.

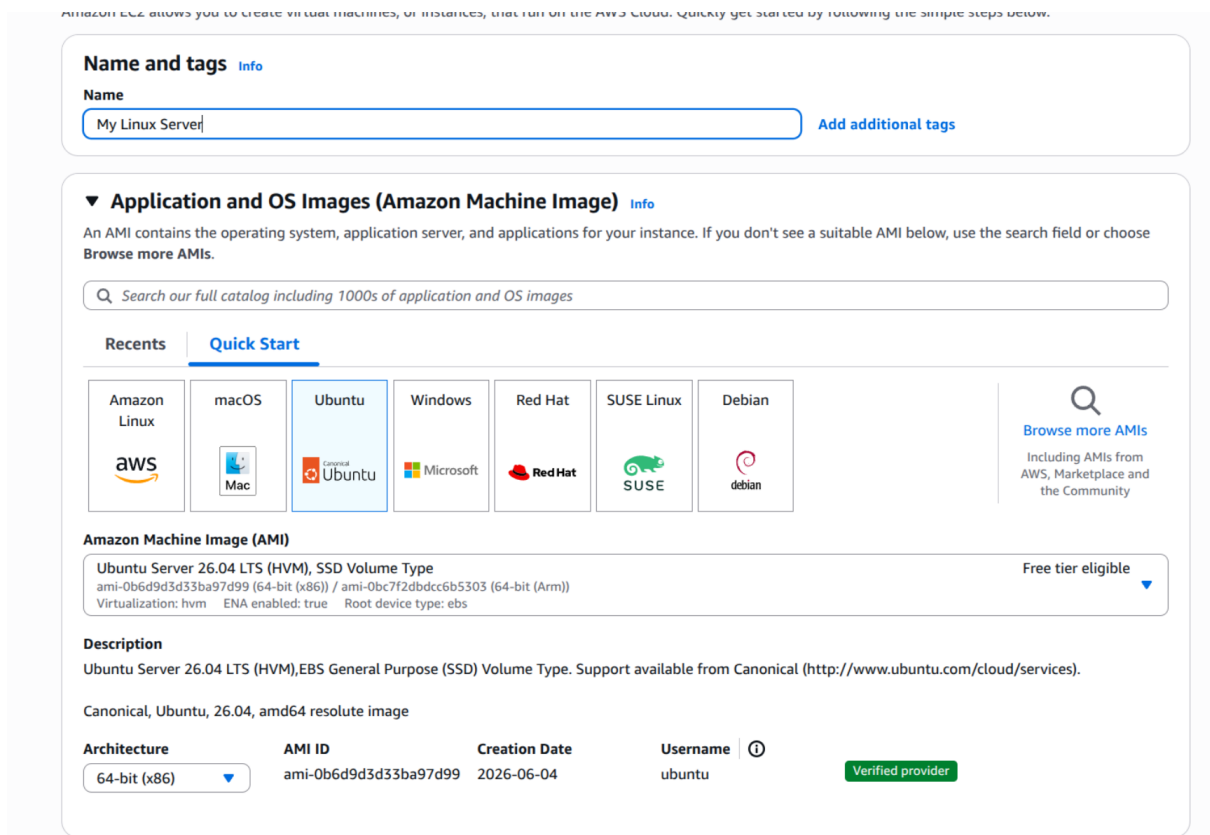


Figure 5: Creating Ubuntu instance

Now select t2.micro for the instance type.

How many vCPUs will the instance have?

How much memory will it have?

How much will it cost per day to run?

If you selected, t2.medium, how much would it cost per day?

If you selected, t2.large, how much would it cost per day?

Now create a new key pair and save it to your local drive. This file contains your private key, and which you will need to connect to your instance. Accept all the other defaults.

Observe the firewall group that will be applied.

Which firewall ports are open on the instance?

What do you think is the main issue with this firewall setting?

How would you change it, once you have created the instance?

Observe the disk storage setting for the instance.

What type of disk will be used? [HDD/SSD]

What do you think is the advantage of using SSD?

For disk storage, what is the default size of the disk that you will create?

What is the maximum storage size for a free tier storage of the AMI instance we are creating?

C.1 Creating the instance

Go ahead and create the instance. Then go back to the AWS Management Console, and find your instance. Wait for it to set its state to running.

Now we will connect to it. For this, we need to create an SSH connection and use the private key we have generated. The public key will be stored on the instance and will authenticate our access. We do not need a username or password to access the instance, as this is often insecure. Our PEM file will give us access (or you can use Putty for the connection).

Now, we will examine the details of our instance (Figure 6). On the instance summary, determine the following:

The public IP address:

The private IP address:

The instance type:

The public IPv4 DNS:

From your local host, can you ping the public IP address? [Yes/No]

Why can't you successfully ping your instance?

Which region of the world is your instance running in?

C.2 Enabling ICMP on firewall

Now, we will enable ICMP on the instance. First, click on the Security tab of the instance summary, and then on the security group.

What is the firewall rule that is applied to the instance?

[SSH/Telnet/FTP/HTTP/HTTPS] for [0.0.0.0/0 or 0.0.0.0/8 or 0.0.0.0/16 or 0.0.0.0/32]

What does 0.0.0.0/0 represent?

Now go ahead and add an ICMP rule for all hosts (Figure 7).

Can you now successfully ping your instance? [Yes/No]

Now, lock your ICMP rule down to just your IP address (you need to use a /32 address for this). Can you still successfully ping the instance? [Yes/No]

Ask your neighbour or one of the lab tutors to ping your instance. Can they successfully ping it? [Yes/No]

What is the advantage of applying the firewall in AWS, rather than in the instance?

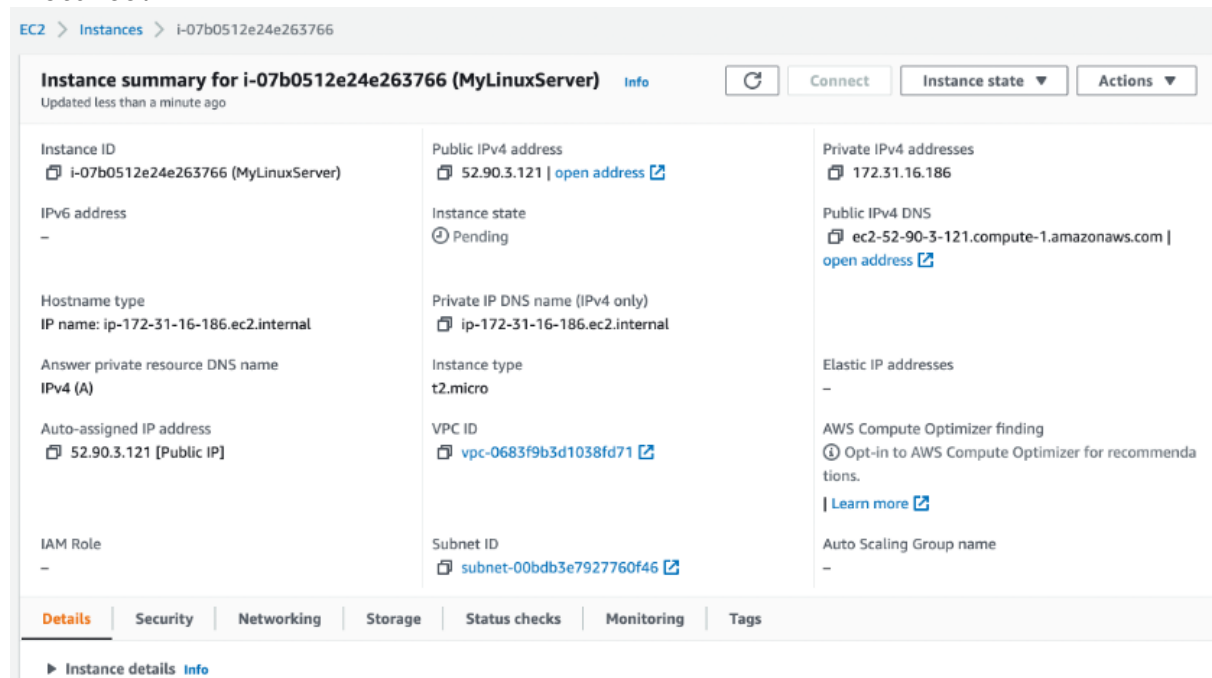


Figure 6: Details of instance

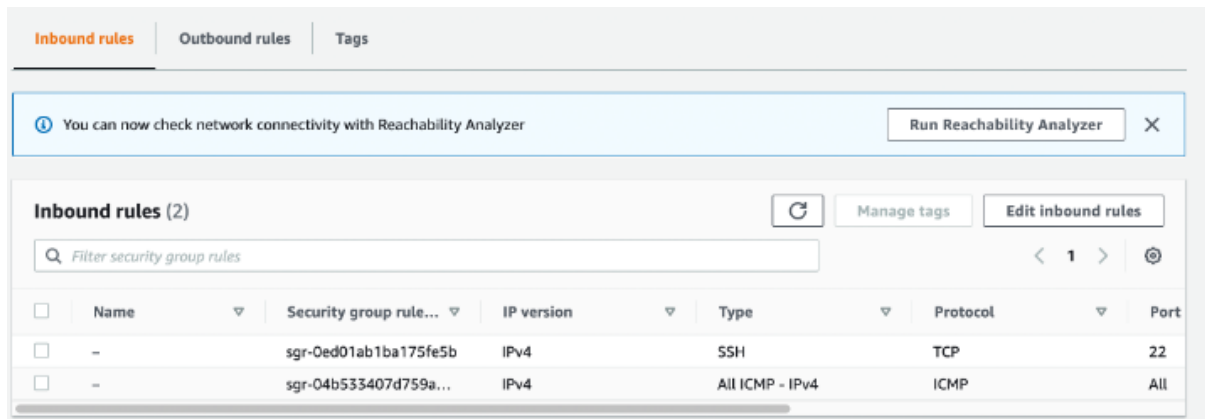


Figure 7: Enable ICMP

C.3 Accessing your instance

Now we will connect to our instance. For this you need SSH (such as provided by OpenSSH). This may be installed on the host you are using (such as in vSoC 2), or from Apps Anywhere. Once you have SSH, press Connect on the summary page, and you should then have tabs for Connect to instance (Figure 8). Next select the SSH client tab, and you will see the details of connecting to your instance with SSH.

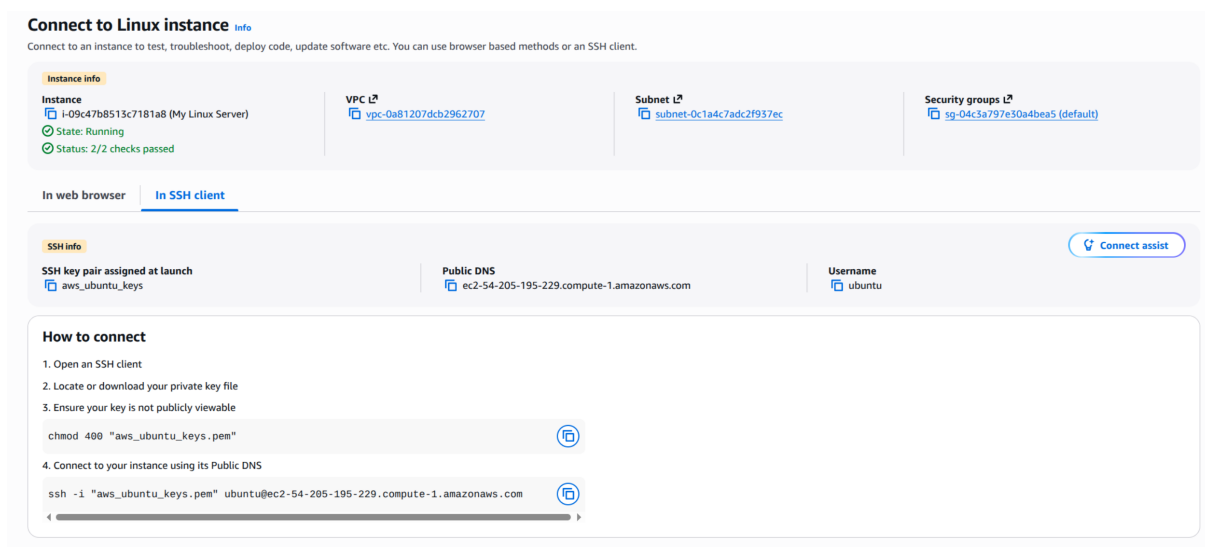


Figure 8: Connect to instance

Now find your PEM file on your local machine (from the command line), and protect it with:

```
chmod 400 "aws_ubuntu_keys.pem"
```

If you are using a Windows console, please see the note in Appendix A.

What protection does this put on your private key?

Next, use the SSH connection with the name of your PEM file and with the DNS (or IP address) for your instance. For example, in the case in Figure 8, we have:

```
ssh -i "aws_ubuntu_keys.pem" ubuntu@ec2-54-205-195-229.compute-1.amazonaws.com
```

What is the name of the user that logs in?

An example of connecting is (In this example it is Amazon Linux Instance, but your Ubuntu should display something similar):

```
% ssh -i "mynewkeypair.pem" ec2-user@ec2-52-90-3-121.compute-1.amazonaws.com
The authenticity of host 'ec2-52-90-3-121.compute-1.amazonaws.com
(52.90.3.121)' can't be established.
ED25519 key fingerprint is
SHA256:/c5UOK6gprKL19XCptNQ1brb9MpYR5wEeqhD/6t+/Wk.
This host key is known by the following other names/addresses:
  ~/.ssh/known_hosts:48: ec2-3-90-189-201.compute-1.amazonaws.com
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'ec2-52-90-3-121.compute-1.amazonaws.com'
(ED25519) to the list of known hosts.
Last login: Fri Sep 30 17:07:00 2022 from ec2-18-206-107-27.compute-1.amazonaws.com
```

```
  _ |  _ |  )
 _ |  ( _ | /  Amazon Linux 2 AMI
 _ | \ _ | _ |
```

```
https://aws.amazon.com/amazon-linux-2/
[ec2-user@ip-172-31-16-186 ~]$
Have you managed to connect? [Yes/No]
```

By using "ip addr show" or "ifconfig" in your instance, what is the private IP address of it?

Can you ping 8.8.8.8 from your instance? [Yes/No]

Is there a folder named .ssh? [Yes/No]

What do you think is the purpose of the file contained in .ssh?

Now create a folder in the top level named "mytestfolder", and put a new file in there named "mytext.txt" (and put some text in this file).

Now go to the EC2 Instance Connect (Figure 9), and press on the Connect button. You should now get a console terminal in the browser.

From your console (Figure 10), verify that your file has been created. Has it been created in the instance? [Yes/No]

Connect to Linux instance [Info](#)

Connect to an instance to test, troubleshoot, deploy code, update software etc. You can use browser based methods or an SSH client.

Instance info

Instance i-09c47b8513c7181a8 (My Linux Server) State: Running Status: 2/2 checks passed	VPC vpc-0a81207dcb2962707	Subnet subnet-0c1a4c7adc2f937ec	Security groups sg-04c3a797e30a4bea5 (default)
---	-------------------------------------	---	--

[In web browser](#) | [In SSH client](#)

Choose how to connect

☒ Public subnet access | [Managed SSH keys](#)
EC2 Instance Connect
Use for instances with public IPs or accessible via standard routing. Requires network connectivity to your instance. Connect using temporary SSH keys managed by AWS.

☐ Private subnet access | [Managed SSH keys](#)
EC2 Instance Connect Endpoint
Use for instances in private subnets. Traffic is routed through a VPC endpoint. Connect using temporary SSH keys managed by AWS.

☐ Maximum security | [IAM authentication](#)
SSM Session Manager
Uses IAM-based authentication. Connect without SSH keys, bastion hosts, or opening any inbound ports.

☐ Direct | [Troubleshoot](#)
EC2 Serial Console
Access your instance's serial port for troubleshooting boot, network, and OS issues. Works without network connectivity. Requires account-level enablement.

Settings

Choose public IP address
☒ IPv4 address (54.205.195.229)
☐ IPv6 address

Username
Enter the username defined in the AMI used to launch the instance. If you didn't define a custom username, use the default username, ec2-user. Read your AMI usage instructions to check if the AMI owner has changed the default AMI username.

Port 22 (SSH) is open to all IPv4 addresses
Port 22 (SSH) is currently open to all IPv4 addresses, indicated by 0.0.0.0/0 in the inbound rule in [your security group](#). For increased security, consider restricting access to only the EC2 Instance Connect service IP addresses for your Region: 18.206.107.24/29. [Learn more](#)

[Cancel](#) | [Connect assist](#) | [Connect](#)

Figure 9: EC2 Instance Connect

```
aws
[Alt+S]
Welcome to Ubuntu 26.04.1 LTS (GNU/Linux 7.0.0-1013-aws x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

System information as of Fri Oct  2 07:19:15 UTC 2026

System load:  0.3          Processes:      116
Usage of /:   42.1% of 6.61GB Users logged in: 0
Memory usage: 23%         IPv4 address for enx0: 172.31.18.160
Swap usage:   0%

 * Ubuntu Pro delivers the most comprehensive open source security and
 * compliance features.
 * https://ubuntu.com/aws/pro

Expanded Security Maintenance for Applications is not enabled.

17 updates can be applied immediately.
4 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

Last login: Thu Oct  1 16:40:09 2026 from 18.206.107.27
ubuntu@ip-172-31-18-160:~$
```

i-09c47b8513c7181a8 (My Linux Server)
PublicIPs: 54.205.195.229 | PrivateIPs: 172.31.18.160

Figure 10: EC2 Instance Connect terminal

Now examine the running services on the instance with:

```
# First you need install the netstat tools
sudo apt install net-tools
# Then type
netstat -l | grep tcp
netstat -l | grep udp
```

Which of the main services are running:

C.4 Installing a Web server

Now we will install a Web server on the instance with:

Ubuntu Commands

```
sudo apt update -y
sudo apt upgrade -y
sudo apt install -y apache2
sudo systemctl start apache2
sudo systemctl enable apache2
```

Next open up a browser on your computer and access your instance for Web access.

Can you connect to it? [Yes/No]

Why can't you connect to it?

Now enable firewall rules on Ports 80 and 443 to allow Web traffic (see Figure 11).

Inbound rules Info							
Security group rule ID	Type Info	Protocol Info	Port range Info	Source Info	Description - optional Info		
sgr-0ed01ab1ba175fe5b	SSH	TCP	22	Custom	0.0.0.0/0		Delete
sgr-04b533407d759a286	All ICMP - IPv4	ICMP	All	Anywh...	0.0.0.0/0	Ping	Delete
-	HTTPS	TCP	443	Anywh...	0.0.0.0/0	All access to Web server	Delete
-	HTTP	TCP	80	Anywh...	0.0.0.0/0	All access to Web server	Delete

Figure 11: Enable HTTP and HTTPs rules

Can you now connect to your Web site? [Yes/No] (see Figure 12)

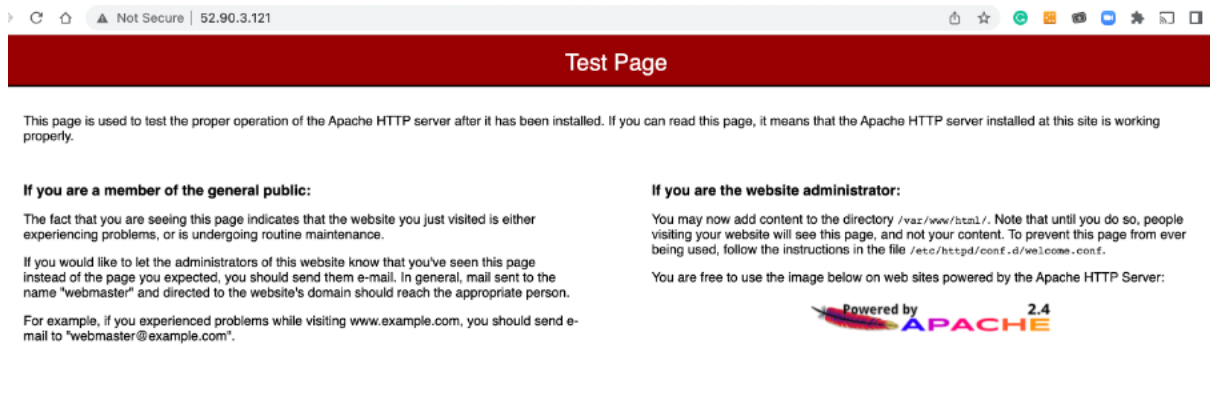


Figure 12: Sample access to the website

Now go into the `/var/www/html` folder, and create a file named “`index.html`”, and add:

```
<h1>Main Web site</h1>
<p>Hello to you</p>
```

And then save the file.

Has it changed the welcome? [Yes/No]

C.6 Auditing

The main logging output is in the `/var/log` folder. Go into this folder and observe some of the files in there. Identify the contents of the following files:

What are the likely contents of the “`auth.log`” file?

What are the likely contents of the “`kern.log`” file?

List the “`/var/log/apache2/access.log`” file in Ubuntu. What are its contents? Can you identify your browser access? (see Figure 13). Which browser type accessed your Web server?

Now try with another browser type (such as Firefox or Chrome) and re-examine the `/var/log/apache2/access.log` file. Did it detect the new browser type?

Now access a file that does not exist in your site (such as `http://AWSIP/test.htm`). Now re-examine the `/var/log/apache2/access.log` file in Ubuntu. What is the status code returned for the access?

```
e/105.0.0.0 Safari/537.36"
187 - - [02/Oct/2022:11:56:24 +0000] "GET /icons/apache_pb2.gif HTTP
34 "http://52.90.3.121/" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_
ebKit/537.36 (KHTML, like Gecko) Chrome/105.0.0.0 Safari/537.36"
187 - - [02/Oct/2022:11:56:24 +0000] "GET /favicon.ico HTTP/1.1" 404
/52.90.3.121/" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) Appl
36 (KHTML, like Gecko) Chrome/105.0.0.0 Safari/537.36"
187 - - [02/Oct/2022:11:58:16 +0000] "-" 408 - "-" "-"
187 - - [02/Oct/2022:12:12:22 +0000] "GET / HTTP/1.1" 200 13 "-" "Mo
acintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Ge
105.0.0.0 Safari/537.36"
187 - - [02/Oct/2022:12:19:53 +0000] "GET / HTTP/1.1" 200 13 "-" "Mo
acintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Ge
105.0.0.0 Safari/537.36"
187 - - [02/Oct/2022:12:19:53 +0000] "GET /favicon.ico HTTP/1.1" 404
/52.90.3.121/" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) Appl
36 (KHTML, like Gecko) Chrome/105.0.0.0 Safari/537.36"
187 - - [02/Oct/2022:12:19:54 +0000] "GET / HTTP/1.1" 304 - "-" "Moz
cintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gec
05.0.0.0 Safari/537.36"
187 - - [02/Oct/2022:12:19:56 +0000] "GET / HTTP/1.1" 304 - "-" "Moz
cintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gec
05.0.0.0 Safari/537.36"
```

Figure 13: Sample list of log/httpd/access_log (you should have something similar and your ubuntu)

C.7 Adding a new user

The ubuntu username can be used to connect back into the server using access authenticated with the private key. We will now create a new user named “napier”, and which can connect to the instance with SSH. For this, we use adduser and passwd on the Linux instance:

```
[ubuntu@ip-172-31-16-186 ~]$ sudo adduser napier
Changing password for user napier.
New password: <yourpass>
Retype new password: <yourpass>
passwd: all authentication tokens updated successfully.
```

Now we will add the new user to the login. For this, we use:

Ubuntu

```
[ubuntu@ip-172-31-16-186 .ssh]$ sudo nano /etc/ssh/sshd_config
Add line of (similar to Figure 15):
AllowUsers ubuntu napier
Change the following to “yes” (see Figure 16):
PasswordAuthentication yes
Comment out the following:
#KbdInteractiveAuthentication no
```

Now restart the SSH service with:

```
[ubuntu@ip-172-31-16-186 .ssh]$ sudo systemctl restart ssh
```

Can you now connect to your instance with the new user and password (but change for the IP address of your instance):

```
ssh napier@54.209.145.85
```

Can you connect with the new user? [Yes/No]

```
e/105.0.0.0 Safari/537.36"
187 - - [02/Oct/2022:11:56:24 +0000] "GET /icons/apache_pb2.gif HTTP
34 "http://52.90.3.121/" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_
ebKit/537.36 (KHTML, like Gecko) Chrome/105.0.0.0 Safari/537.36"
187 - - [02/Oct/2022:11:56:24 +0000] "GET /favicon.ico HTTP/1.1" 404
/52.90.3.121/" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) Appl
36 (KHTML, like Gecko) Chrome/105.0.0.0 Safari/537.36"
187 - - [02/Oct/2022:11:58:16 +0000] "-" 408 - "-" "-"
187 - - [02/Oct/2022:12:12:22 +0000] "GET / HTTP/1.1" 200 13 "-" "Mo
acintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Ge
105.0.0.0 Safari/537.36"
187 - - [02/Oct/2022:12:19:53 +0000] "GET / HTTP/1.1" 200 13 "-" "Mo
acintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Ge
105.0.0.0 Safari/537.36"
187 - - [02/Oct/2022:12:19:53 +0000] "GET /favicon.ico HTTP/1.1" 404
/52.90.3.121/" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) Appl
36 (KHTML, like Gecko) Chrome/105.0.0.0 Safari/537.36"
187 - - [02/Oct/2022:12:19:54 +0000] "GET / HTTP/1.1" 304 - "-" "Moz
cintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gec
05.0.0.0 Safari/537.36"
187 - - [02/Oct/2022:12:19:56 +0000] "GET / HTTP/1.1" 304 - "-" "Moz
cintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gec
05.0.0.0 Safari/537.36"
```

Figure 14: Accessing instances

```
# $OpenBSD: sshd_config,v 1.100 2016/08/15 12:32:04 naddy Exp $
AllowUsers ec2-user napier
# This is the sshd server system-wide configuration file. See
# sshd_config(5) for more information.

# This sshd was compiled with PATH=/usr/local/bin:/usr/bin

# The strategy used for options in the default sshd_config shipped with
```

Figure 15: Accessing instances

```
# To disable tunneled clear text passwords, change to no here!
#PasswordAuthentication yes
#PermitEmptyPasswords no
PasswordAuthentication yes

# Change to no to disable s/key passwords
#ChallengeResponseAuthentication yes
ChallengeResponseAuthentication no
```

Figure 16: Accessing instances

C.8 Accessing from AWS prompt

We can also access our instance from the AWS terminal prompt. For this return to your AWS Academy console, and enter the command (Figure 17):

```
$ aws ec2 describe-instances
```

From the results, can you identify the following:

Instance type:

Public IP address:

Private IP address:

State:



```
ddd_v1_w_W3n_1455598@runweb63093:~$ aws ec2 describe-instances
{
  "Reservations": [
    {
      "Groups": [],
      "Instances": [
        {
          "AmiLaunchIndex": 0,
          "ImageId": "ami-026b57f3c383c2eec",
          "InstanceId": "i-07b0512e24e263766",
          "InstanceType": "t2.micro",
          "KeyName": "mynewkeypair",
          "LaunchTime": "2022-10-02T11:14:16+00:00",
          "Monitoring": {
            "State": "disabled"
          },
          "Placement": {
            "AvailabilityZone": "us-east-1b",
            "GroupName": "",
            "Tenancy": "default"
          },
          "PrivateDnsName": "ip-172-31-16-186.ec2.internal",
          "PrivateIpAddress": "172.31.16.186",
          "ProductCodes": [],
          "PublicDnsName": "ec2-52-90-3-121.compute-1.amazonaws.com",
          "PublicIpAddress": "52.90.3.121",
          "State": {
            "Code": 16,
            "Name": "running"
          },
          "StateTransitionReason": "",
          "SubnetId": "subnet-00bdb3e7927760f46",
          "VpcId": "vpc-0683f9b3d1038fd71",
          "Architecture": "x86_64",
```

Figure 17: Describe instances

Now we will stop our instance using an AWS EC2 command. Run the following with your instance ID (see Figure 18):

```
aws ec2 stop-instances --instance-ids [My-instance-ID]
```

From the AWS Management Console, has your instance stopped? [Yes/No]

```
ddd_v1_w_w3n_1455598@runweb62964:~$ aws ec2 stop-instances --instance-ids i-07b0512e24e263766
{
  "StoppingInstances": [
    {
      "CurrentState": {
        "Code": 64,
        "Name": "stopping"
      },
      "InstanceId": "i-07b0512e24e263766",
      "PreviousState": {
        "Code": 16,
        "Name": "running"
      }
    }
  ]
}
ddd_v1_w_w3n_1455598@runweb62964:~$ █
```

Figure 18: Stopping an instance

Now we will restart the instance with:

```
aws ec2 start-instances --instance-ids [My-instance-ID]
```

Has the instance re-started? [Yes/No]

Now we will change the instance type from t3.micro to t3.small. To do this, run the following commands:

```
aws ec2 stop-instances --instance-ids [My-instance-ID]
aws ec2 wait instance-stopped --instance-ids [My-instance-ID]
aws ec2 modify-instance-attribute --instance-id [My-instance-ID] --instance-type "{\"Value\": \"t3.small\"}"
aws ec2 start-instances --instance-ids [My-instance-ID]
```

Did it change the instance type? [Yes/No]

Can you still get access to your instance?

By observing the script and investigating what t3.micro and t3.small are, can you determine what has changed about your instance?

Now, revert the instance to t3.micro and suspend it.

D Creating and Securing a Windows 2022 Server

In this part of the lab, we will create a Windows 2022 server instance with t3.micro (note, that this is very low for vCPUs and memory, so the performance may be a little lacking). First create a new instance, and give it a name, such as “MyWindowsServer” (Figure 19).

Launch an instance [Info](#)

Amazon EC2 allows you to create virtual machines, or instances, that run on the AWS Cloud. Quickly get started by following the simple steps below.

Name and tags [Info](#)

Name

[Add additional tags](#)

▼ Application and OS Images (Amazon Machine Image) [Info](#)

An AMI contains the operating system, application server, and applications for your instance. If you don't see a suitable AMI below, use the search field or choose [Browse more AMIs](#).

Recents | **Quick Start**

Amazon Linux

macOS

Ubuntu

Windows

Red Hat

SUSE Linux

Debian

[Browse more AMIs](#)
Including AMIs from AWS, Marketplace and the Community

Amazon Machine Image (AMI)

Microsoft Windows Server 2022 Base

ami-0ed58a129008c2cc2 (64-bit (x86))

Virtualization: hvm ENA enabled: true Root device type: ebs

Free tier eligible

Description

Microsoft Windows 2022 Datacenter edition.

Microsoft Windows Server 2022 Full Locale English AMI provided by Amazon

Architecture	AMI ID	Creation Date	Username
64-bit (x86)	ami-0ed58a129008c2cc2	2026-09-18	Administrator

[Verified provider](#)

Figure 19: Creating Windows 2022 instance

Now select t2.micro for the instance type.

How many vCPUs will the instance have?

How much memory will it have?

How much will it cost per day to run?

If you selected, t2.medium, how much would it cost per day?

If you selected t2.large, how much would it cost per day?

Now create a new key pair and save it to your local drive. This file contains your private key, and which you will need to connect to your instance. Accept all the other defaults.

Observe the firewall group that will be applied.

Which firewall ports are open on the instance?

What is the main issue with this firewall setting?

How would you change it, once you have created the instance?

Observe the disk storage setting for the instance.

What type of disk will be used? [HDD/SSD]

What is the advantage of using SSD?

For disk storage, what is the size of the disk that you will create?

What is the maximum storage size for a free tier storage of the AMI instance we are creating?

D.1 Creating the instance

Go ahead and create the instance. Go back to the Management Console and find your instance. Wait for it to set its state to running. Now we will connect to it. For this, we need to create an RDP connection and use the private key we have generated to generate the initial password.

Now, we will examine the details of our instance (Figure 20). On the instance summary, determine the following:

The public IP address:

The private IP address:

The instance type:

The public IPv4 DNS:

From your local host, can you ping the public IP address? [Yes/No]

Why can't you successfully ping your instance?

Which region of the world is your instance running in?

D.2 Enabling ICMP on firewall

Now we will enable ICMP on the instance. First click on the Security tab of the instance summary, and then on the security group.

What is the firewall rule that is applied to the instance?

[SSH/RDP/Telnet/FTP/HTTP/HTTPS] for [0.0.0.0/0 or 0.0.0.0/8 or 0.0.0.0/16 or 0.0.0.0/32]

What does 0.0.0.0/0 represent?

Now go ahead and add an ICMP rule for all hosts (Figure 121).

Can you now successfully ping your instance? [Yes/No]

We will not be able to ping the instance yet, as the firewall on Windows is disabling it.

Instance summary for i-07d723258364f7172 (MyWindowsServer)
[Info](#)

[Connect](#)
[Instance state ▼](#)
[Actions ▼](#)

Updated less than a minute ago

Instance ID i-07d723258364f7172 (MyWindowsServer)	Public IPv4 address 54.83.154.0 open address	Private IPv4 addresses 172.31.85.24
IPv6 address -	Instance state Running	Public IPv4 DNS ec2-54-83-154-0.compute-1.amazonaws.com open address
Hostname type IP name: ip-172-31-85-24.ec2.internal	Private IP DNS name (IPv4 only) ip-172-31-85-24.ec2.internal	
Answer private resource DNS name IPv4 (A)	Instance type t2.micro	Elastic IP addresses -
Auto-assigned IP address 54.83.154.0 [Public IP]	VPC ID vpc-0683f9b3d1038fd71	AWS Compute Optimizer finding Opt-in to AWS Compute Optimizer for recommendations. Learn more
IAM Role -	Subnet ID subnet-07a3be17bcc59528b	Auto Scaling Group name -

[Details](#)
[Security](#)
[Networking](#)
[Storage](#)
[Status checks](#)
[Monitoring](#)
[Tags](#)

Figure 20: Details of instance

Inbound rules
[Info](#)

Security group rule ID	Type Info	Protocol Info	Port range Info	Source Info	Description - optional Info	
sgr-0101b70fb7c795a97	RDP ▼	TCP	3389	Custom ▼ 0.0.0.0/0		Delete
-	Custom ICMP - IPv4 ▼	All ▼	All	Anywh... ▼ 0.0.0.0/0	Ping All	Delete

Figure 21: Enable ICMP

D.3 Accessing your instance

Now we will connect to our instance. For this, you need RDP. Next Connect to the instance (Figure 22). Click on “Get password” and present your PEM file, and it should reveal the password (Figure 23).

Connection Type

☒ **Connect using RDP client**
Download a file to use with your RDP client and retrieve your password.

☐ **Connect using Fleet Manager**
To connect to the instance using Fleet Manager Remote Desktop, the SSM Agent must be installed and running on the instance. For more information, see [Working with SSM Agent](#)

You can connect to your Windows instance using a remote desktop client of your choice, and by downloading and running the RDP shortcut file below:

Download remote desktop file

When prompted, connect to your instance using the following details:

Public DNS ec2-54-83-154-0.compute-1.amazonaws.com	User name Administrator
Password 7s8l-G?Jzzh;Lb8%KyGdmn6JwqReoRF-	

Figure 22: Connect to instance

Session Manager | **RDP client** | EC2 serial console

Instance ID
 i-07d723258364f7172 (MyWindowsServer)

Connection Type

☒ **Connect using RDP client**
Download a file to use with your RDP client and retrieve your password.

☐ **Connect using Fleet Manager**
To connect to the instance using Fleet Manager Remote Desktop, the SSM Agent must be installed and running on the instance. For more information, see [Working with SSM Agent](#)

You can connect to your Windows instance using a remote desktop client of your choice, and by downloading and running the RDP shortcut file below:

Download remote desktop file

When prompted, connect to your instance using the following details:

Public DNS ec2-54-83-154-0.compute-1.amazonaws.com	User name Administrator
Password Get password	

Figure 23: Reveal password

Have you managed to connect? [Yes/No] (Figure 24)

18

By using “ipconfig” in your instance, what is the private IP address of it?

Can you ping 8.8.8.8 from your instance? [Yes/No]

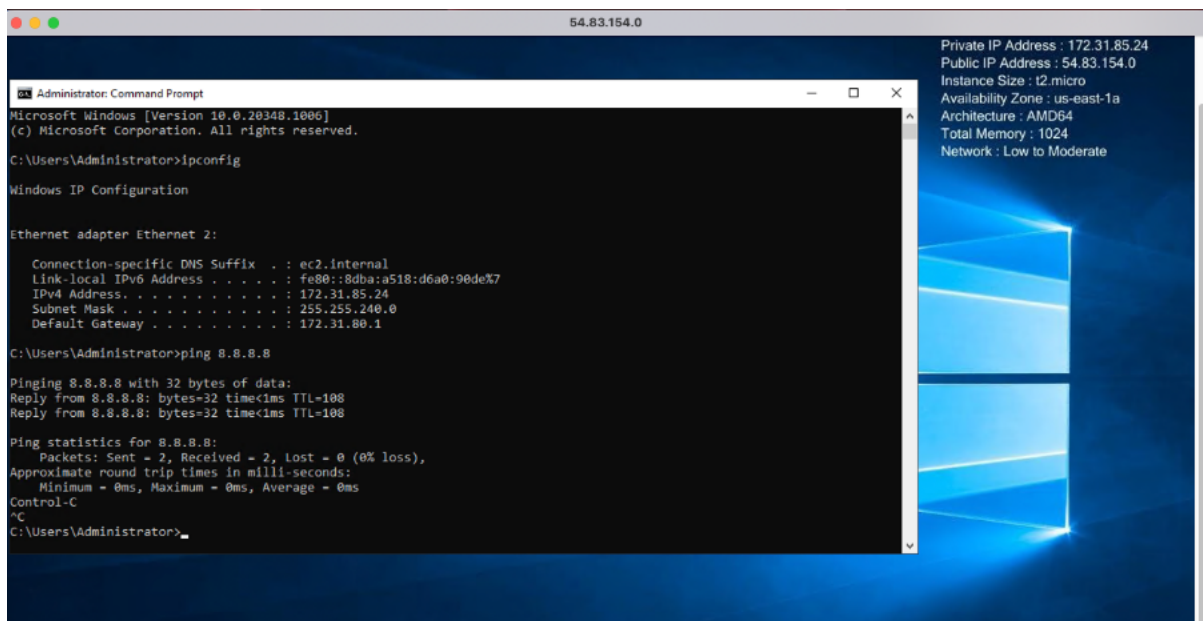


Figure 24: Windows 2022

D.4 Enable ICMP on instance

We have enabled the AWS firewall for ICMP. Now we will open up ICMP in the instance. For this open-up with Advanced Windows firewall, and enable the rule for “File and Printer Sharing (ICMP-in)” – as shown in Figure 25.

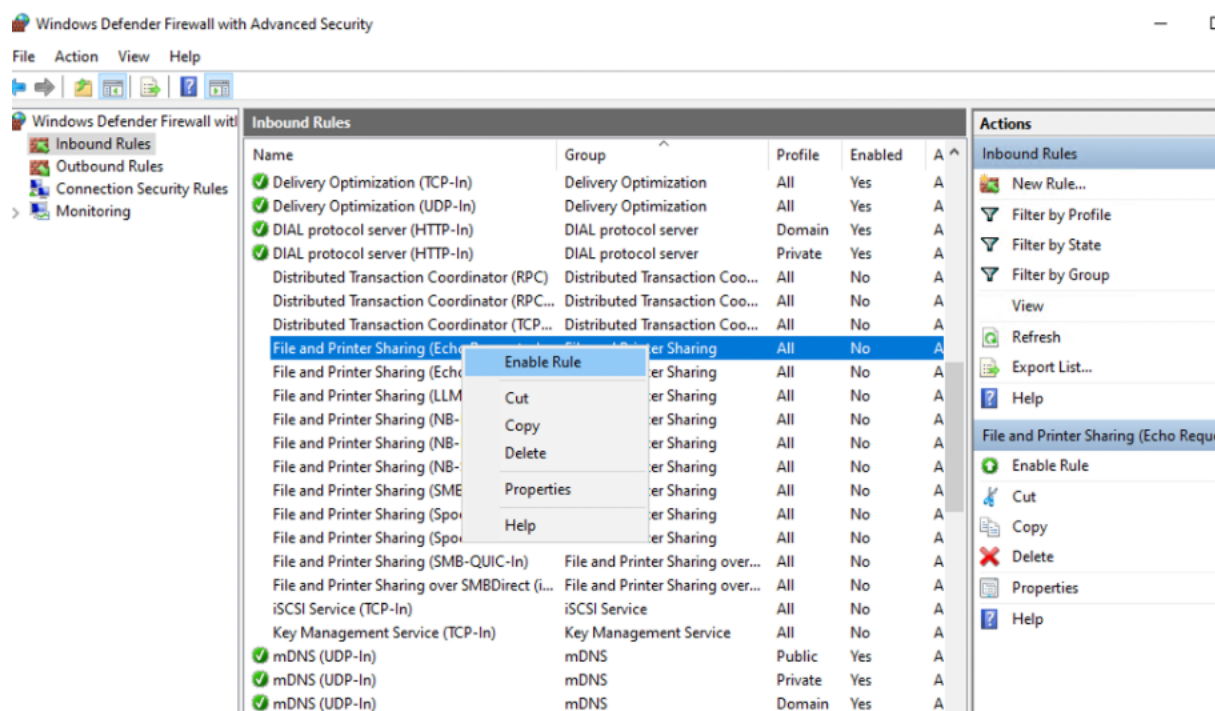


Figure 25: Enable ICMP

Can you successfully ping the instance from your instance? [Yes/No]

D.5 Show running services

Now examine the running services on the instance with:

```
$ netstat -i | grep tcp $ netstat -i | grep udp
```

Which of the main services are running:

D.6 Enable Web server

Now select Server Manage, and “Add a Role” for Web Server (IIS) (Figure 26).

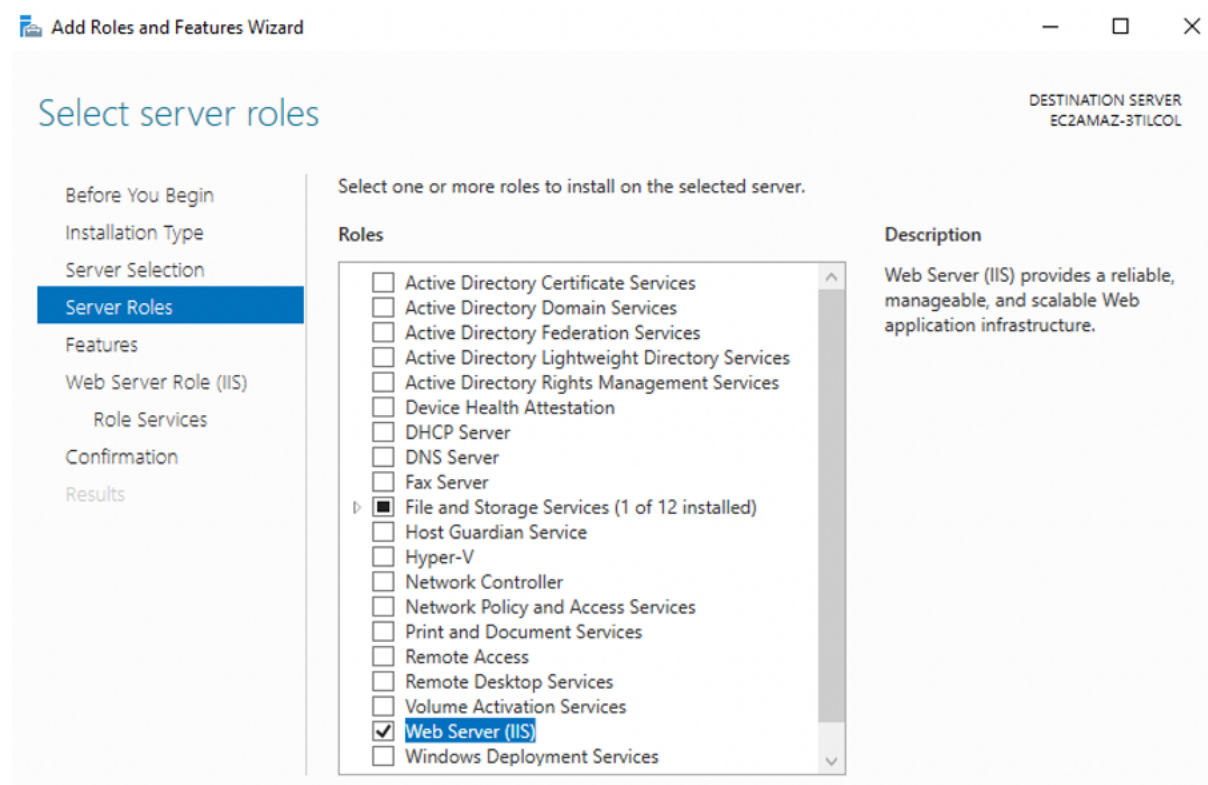


Figure 26: Enable ICMP

Now open a browser on the instance, and access <http://localhost>

Can you connect to the IIS Web server? [Yes/No] (see Figure 25)

Now open up your AWS firewall for Port 80 (Figure 27).

Inbound rules Info						
Security group rule ID	Type Info	Protocol Info	Port range Info	Source Info	Description - optional Info	
sgr-088cf5bfd7b0741f4	All ICMP - IPv4	ICMP	All	Custom	0.0.0.0/0	Ping All Delete
sgr-0101b70fb7c795a97	RDP	TCP	3389	Custom	0.0.0.0/0	Delete
-	HTTP	TCP	80	Anywh...	0.0.0.0/0	Web Delete

Figure 27: Enable HTTP

Now open a browser on the instance, and access [http://\[IP of AWS\]](http://[IP of AWS])

Can you connect to the IIS Web server? [Yes/No] (Figure 29)

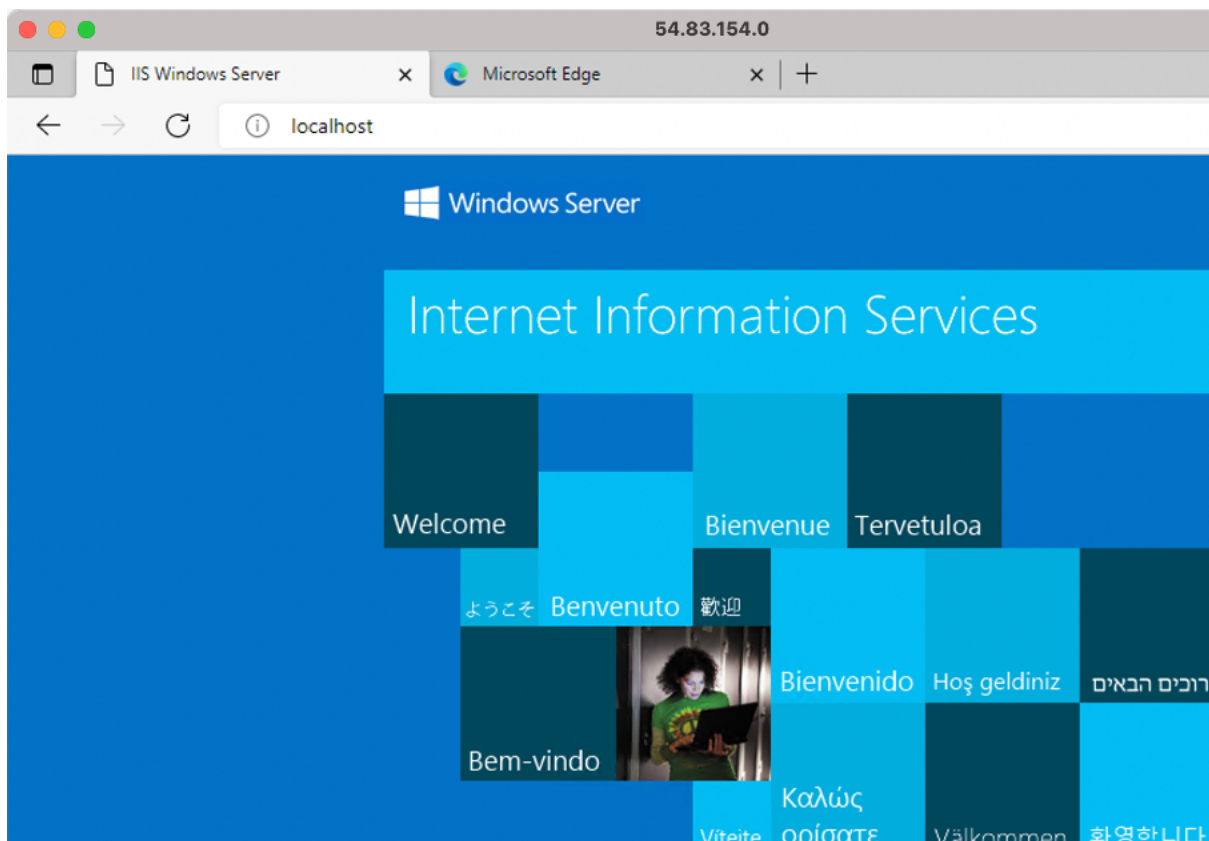


Figure 28: Local host

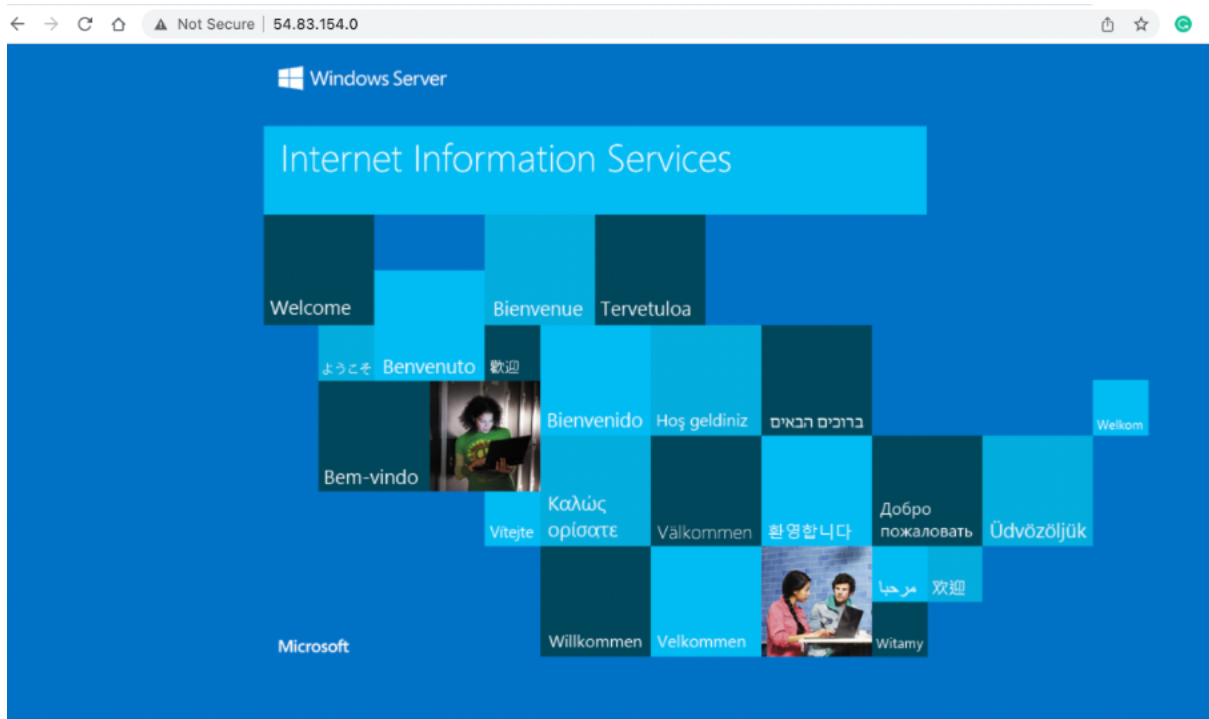


Figure 29: Remote access

Now go into the `c:\inetpub\wwwroot` folder, and create a file named “`iisstart.html`”, and add:

```
<h1>Main Web site</h1>
<p>Hello to you</p>
```

And then save the file.

Has it changed the welcome? [Yes/No]

D.7 Auditing

The main logging output is in the “`C:\inetpub\logs\LogFiles\W3SVC1`” folder. Identify the contents of the following files:

Go into the “`C:\inetpub\logs\LogFiles\W3SVC1`” folder, and list the file in there. What are its contents? Can you identify your browser access? Which browser type accessed your Web server?

Now try with another browser type, and re-examine the `log/httpd/access_log` file. Did it detect the new browser type?

Now access a file that does not exist in your site (such as `http://AWSIP/test.htm`). Now re-examine the file. What is the status code returned for the access?

D.8 Changing Administrator password

We can change the Administrator password, with something like:

```
net user administrator mynewpassword$$7k1
```

Or set up your own

```
net user administrator *
```

2.1 E Python Access

Your unique account will have been generated, and you can access it with `aws_access_key_id` and `aws_secret_access_key` (from AWS details). You will also find that your console has been set up with the details already set up for you. For this, there is a hidden folder named `.aws`, and there is a file named `credentials` in there:

(You need the CLI in the main AWS Academy Learner Lab to find this)

≡ ALLv2EN-US-LT113-189977 > Modules > AWS Academy Learner Lab > Launch AWS Academy Learner Lab

The screenshot shows the AWS Academy Learner Lab interface. The top bar indicates 'Used \$0.3 of \$50' and '03:19'. The main terminal window shows the following output:

```
eee@W.6848988@runweb251800:~$ ls -al
total 36
drwxr-xr-x 5 eee@W.6848988 apache 6144 Sep 28 09:40 .
drwxr-xr-x 5 eee@W.6848988 apache 6144 Sep 18 02:21 ..
dr-xr-xr-x 2 eee@W.6848988 apache 6144 Sep 18 02:23 .aws
-rw-r--r-- 1 eee@W.6848988 eee 5 3193412 6128 Sep 28 09:43 .cdt
-rw-r--r-- 1 eee@W.6848988 apache 57 Sep 18 02:21 .gitconfig
drwxr-xr-x 2 eee@W.6848988 apache 6144 Sep 18 02:23 .ssh
-rw-r--r-- 1 root root 3883 Sep 29 04:58 .termrc
dr-xr-xr-x 2 root root 6144 Sep 18 02:21 .voc
eee@W.6848988@runweb251800:~$ cd .aws
eee@W.6848988@runweb251800:~/.aws$ ls -al
total 16
dr-xr-xr-x 2 eee@W.6848988 apache 6144 Sep 18 02:23 .
drwxr-xr-x 5 eee@W.6848988 apache 6144 Sep 28 09:40 ..
-r--r--r-- 1 eee@W.6848988 apache 29 Sep 29 04:32 config
-r--r--r-- 1 eee@W.6848988 apache 901 Sep 29 04:32 credentials
eee@W.6848988@runweb251800:~/.aws$ cat credentials
[default]
aws_access_key_id = ASIAIY3WUGAKHEPPYGT
aws_secret_access_key = MeedSL1KbY1hbu0efvF++Q2KGJHED9qYw-Int
aws_session_token = IqoJb3Jp2Z1uX2VjE1T/////////wEaCXVZLXdlc3QML3
IWEYCIQD0ueTvQ7yhbecSOLHX1c2L2f/mFWC7J0ZqB9s8bcYwThAjaBvdc1X9J
BPYjGDN9wmsFP9B3SFxyQ/6n1JFCQyqkqCCEQARuNNtXwJY50TESMjUy1gz/c
KMPPr7uP9h6sqhKqAvrYsAcuG1u1d/qvFTB8P1Z1Wb51w+J84Skb1W0Kx70
KPVYKhyeZ1u1LHQ7ZrBjR1R1J3Z1Nfpp1X7p8XQ9s8p5m1J0Tn1Acu070qQ
xxjYXIS8DLK5ZnaXv5DyckfZVHjwkft+Yuw+700qLMB6A1431Qm751BuzR6ZB
ZFmY7LE7rVJ63mW5G7EMGfEnU/Ye1AknUXZarVku05Rpa1TbJbYrb+0f0/6y
7xV5x0R7Q2p0Kaw1++gocVNH29cV59BQURM0BLIjH+g6SK7nGBvYHEMyg/ek
1oZ2dM6E4n1eLkFTFTFPF9V1JL2J25y0s1D1VZCryMLD7LUG0pWmDZ2F8B20B
F9Mba1ky2YKZLJfS/5+0+12Mj4P/usHhde1+s14Vb0/YuWCDLWY+P7Yy03Bn4
dZD+R3B31GdwyFkn/Ddyz3TAyhblXz+66ZgU7K2vMWHK9uZ1q454xorZZUxa2G0
JHr21F0H/c68KjnmHqKaneqCw/SN5c2FT/jvghcyk3bMNCvDz8q+ryfKb+bmV/76u8c
eee@W.6848988@runweb251800:~/.aws$
```

The 'Cloud Access' panel on the right shows the 'AWS CLI' section with the instruction 'Copy and paste the following into ~/.aws/credentials' and a text area containing the same credentials as shown in the terminal output.

```
ddd_v1_w_W3n_1455598@runweb63277:~$ ls -al
drwxrwx--- 5 ddd_v1_w_W3n_1455598 apache 6144 Oct 2 10:13 .
drwxrwx--- 5 ddd_v1_w_W3n_1455598 apache 6144 Sep 29 10:32 ..
dr-xr-xr-x 2 ddd_v1_w_W3n_1455598 apache 6144 Sep 29 12:08 .aws
-rw-rw-r-- 1 ddd_v1_w_W3n_1455598 apache 51 Sep 29 10:32 .gitconfig
drwxr-x--- 2 ddd_v1_w_W3n_1455598 apache 6144 Sep 29 12:08 .ssh
-rw-r--r-- 1 root root 3851 Oct 4 02:44 .termrc
dr-xr-xr-x 2 root root 6144 Sep 29 10:32 .voc
ddd_v1_w_W3n_1455598@runweb63277:~$ cd .aws
ddd_v1_w_W3n_1455598@runweb63277:~/.aws$ ls -al
dr-xr-xr-x 2 ddd_v1_w_W3n_1455598 apache 6144 Sep 29 12:08 .
drwxrwx--- 5 ddd_v1_w_W3n_1455598 apache 6144 Oct 2 10:13 ..
-r--r--r-- 1 ddd_v1_w_W3n_1455598 apache 29 Oct 4 00:19 config
-r--r--r-- 1 ddd_v1_w_W3n_1455598 apache 501 Oct 4 00:19 credentials
ddd_v1_w_W3n_1455598@runweb63277:~/.aws$ cat credentials
```

List the contents of the `credentials` file, and verify that it contains the same credentials from the AWS details button.

Are they the same? [Yes/No]

Now create a Python file which will show your instances in the terminal window (such as `1.py`):

To create a file:

```
cd ~ # brings you back to home - no sudo needed here!  
nano 1.py
```

Then type or paste the code below. (Remember to use Ctrl+Shift+V to paste anything in terminal or nano)

```
import boto3  
ec2 = boto3.client('ec2', region_name='us-east-1')  
print(ec2.describe_instances())
```

Then Ctrl+O, then enter to save, then Ctrl+X to exit.



Figure: Python file creation

Save the file, and then run the file with Python3 and prove that it shows your instances (Figure).

```
python3 1.py
```

```

eee.W.6848988@runweb251809:~$ nano 1.py
eee.W.6848988@runweb251809:~$ python3 1.py
/usr/lib/python3.7/site-packages/boto3/compat.py:82: PythonDeprecationWarning: Boto3 will no longer support Python 3.7 starting December 13, 2023. To continue receiving serv
ice updates, bug fixes, and security updates please upgrade to Python 3.8 or later. More information can be found here: https://aws.amazon.com/blogs/developer/python-support
-policy-updates-for-aws-sdks-and-tools/
  warnings.warn(warning, PythonDeprecationWarning)
{'Reservations': [{'Groups': [], 'Instances': [{'AmiLaunchIndex': 0, 'ImageId': 'ami-0ed58a12908c2cc2', 'InstanceId': 'i-03699d285948bc881', 'InstanceType': 't2.micro', 'Ke
yName': 'win2keys', 'LaunchTime': datetime.datetime(2026, 9, 29, 11, 32, 4, tzinfo=tzutc()), 'Monitoring': {'State': 'disabled'}, 'Placement': {'AvailabilityZone': 'us-east
-1b', 'GroupName': '', 'Tenancy': 'default'}, 'Platform': 'windows', 'PrivateDnsName': 'ip-172-31-16-148.ec2.internal', 'PrivateIpAddress': '172.31.16.148', 'ProductCodes':
[], 'PublicDnsName': 'ec2-34-230-31-18.compute-1.amazonaws.com', 'PublicIpAddress': '34.230.31.18', 'State': {'Code': 16, 'Name': 'running', 'StateTransitionReason': '', 'S
ubnetId': 'subnet-0c1a4c7adc2f937ec', 'VpcId': 'vpc-0a81207dcb2962707', 'Architecture': 'x86_64', 'BlockDeviceMappings': [{'DeviceName': '/dev/sda1', 'Ebs': {'AttachTime': d
atetime.datetime(2026, 9, 29, 7, 48, 38, tzinfo=tzutc()), 'DeleteOnTermination': True, 'Status': 'attached', 'VolumeId': 'vol-01d359ab9fc0aa831'}}], 'ClientToken': '7802d218
-7597-4d54-9259-1649b6c38586', 'EbsOptimized': False, 'EnaSupport': True, 'Hypervisor': 'xen', 'NetworkInterfaces': [{'Association': {'IpOwnerId': 'amazon', 'PublicDnsName':
'ec2-34-230-31-18.compute-1.amazonaws.com', 'PublicIp': '34.230.31.18'}, 'Attachment': {'AttachTime': datetime.datetime(2026, 9, 29, 7, 48, 37, tzinfo=tzutc()), 'Attachment
Id': 'eni-attach-8cb7ff31166a7532c', 'DeleteOnTermination': True, 'DeviceIndex': 0, 'Status': 'attached', 'NetworkCardIndex': 0}, 'Description': '', 'Groups': [{'GroupName':
'default', 'GroupId': 'sg-04c3a797e30a4bea5'}], 'Ipv6Addresses': [], 'MacAddress': '0a:ff:ca:f5:02:e9', 'NetworkInterfaceId': 'eni-0cbb2338e67613933', 'OwnerId': '590269919
252', 'PrivateDnsName': 'ip-172-31-16-148.ec2.internal', 'PrivateIpAddress': '172.31.16.148', 'PrivateIPAddresses': [{'Association': {'IpOwnerId': 'amazon', 'PublicDnsName':
'ec2-34-230-31-18.compute-1.amazonaws.com', 'PublicIp': '34.230.31.18'}, 'Primary': True, 'PrivateDnsName': 'ip-172-31-16-148.ec2.internal', 'PrivateIpAddress': '172.31.16
.148'}], 'SourceDestCheck': True, 'Status': 'in-use', 'SubnetId': 'subnet-0c1a4c7adc2f937ec', 'VpcId': 'vpc-0a81207dcb2962707', 'InterfaceType': 'interface'}, 'RootDeviceNam
e': '/dev/sda1', 'RootDeviceType': 'ebs', 'SecurityGroups': [{'GroupName': 'default', 'GroupId': 'sg-04c3a797e30a4bea5'}], 'SourceDestCheck': True, 'Tags': [{'Key': 'Name',
'Value': 'My Windows 22'}], 'VirtualizationType': 'hvm', 'CpuOptions': {'CoreCount': 1, 'ThreadsPerCore': 1}, 'CapacityReservationSpecification': {'CapacityReservationPrefe
rence': 'open'}, 'HibernationOptions': {'Configured': False}, 'MetadataOptions': {'State': 'applied', 'HttpTokens': 'required', 'HttpPutResponseHopLimit': 2, 'HttpEndpoint':
'enabled', 'HttpProtocolIpv6': 'disabled', 'InstanceMetadataTags': 'disabled'}, 'EnclaveOptions': {'Enabled': False}, 'PlatformDetails': {'Windows': 'UsageOperation': 'RunIn
stances:0902', 'UsageOperationUpdateTime': datetime.datetime(2026, 9, 29, 7, 48, 37, tzinfo=tzutc()), 'PrivateDnsNameOptions': {'HostNameType': 'ip-name', 'EnableResourceNam
eDnsRecord': True, 'EnableResourceNameDnsAAAARecord': False}, 'MaintenanceOptions': {'AutoRecovery': 'default'}, 'CurrentInstanceBootMode': 'legacy-bios'}, 'OwnerId': '590
269919252', 'ReservationId': 'r-0b9e346b4bc2b9dc', 'Groups': [], 'Instances': [{'AmiLaunchIndex': 0, 'ImageId': 'ami-096d9d3d33ba97d99', 'InstanceId': 'i-09c47b8513c7381a
8', 'InstanceType': 't2.micro', 'KeyName': 'aws_ubuntu_keys', 'LaunchTime': datetime.datetime(2026, 9, 29, 11, 32, 4, tzinfo=tzutc()), 'Monitoring': {'State': 'disabled'}, 'P
lacement': {'AvailabilityZone': 'us-east-1b', 'GroupName': '', 'Tenancy': 'default'}, 'PrivateDnsName': 'ip-172-31-18-160.ec2.internal', 'PrivateIpAddress': '172.31.18.160',
'ProductCodes': [], 'PublicDnsName': 'ec2-34-233-134-52.compute-1.amazonaws.com', 'PublicIpAddress': '34.233.134.52', 'State': {'Code': 16, 'Name': 'running', 'StateTrans
itionReason': '', 'SubnetId': 'subnet-0c1a4c7adc2f937ec', 'VpcId': 'vpc-0a81207dcb2962707', 'Architecture': 'x86_64', 'BlockDeviceMappings': [{'DeviceName': '/dev/sda1', 'Eb
s': {'AttachTime': datetime.datetime(2026, 9, 28, 15, 42, 47, tzinfo=tzutc()), 'DeleteOnTermination': True, 'Status': 'attached', 'VolumeId': 'vol-05e0efb082092db19'}], 'Cl
ientToken': '669a6917-95ad-4b4c-87d1-3d4720c6eb3', 'EbsOptimized': False, 'EnaSupport': True, 'Hypervisor': 'xen', 'NetworkInterfaces': [{'Association': {'IpOwnerId': 'amaz
on', 'PublicDnsName': 'ec2-34-233-134-52.compute-1.amazonaws.com', 'PublicIp': '34.233.134.52'}, 'Attachment': {'AttachTime': datetime.datetime(2026, 9, 28, 15, 42, 46, tzin
fo=tzutc()), 'AttachmentId': 'eni-attach-079f168cbd33ef444', 'DeleteOnTermination': True, 'DeviceIndex': 0, 'Status': 'attached', 'NetworkCardIndex': 0}, 'Description': '',
'Groups': [{'GroupName': 'default', 'GroupId': 'sg-04c3a797e30a4bea5'}], 'Ipv6Addresses': [], 'MacAddress': '0a:ff:f8:2e:f6:65', 'NetworkInterfaceId': 'eni-0d5157eddfa8ecb44
', 'OwnerId': '590269919252', 'PrivateDnsName': 'ip-172-31-18-160.ec2.internal', 'PrivateIpAddress': '172.31.18.160', 'PrivateIPAddresses': [{'Association': {'IpOwnerId': 'a
amazon', 'PublicDnsName': 'ec2-34-233-134-52.compute-1.amazonaws.com', 'PublicIp': '34.233.134.52'}, 'Primary': True, 'PrivateDnsName': 'ip-172-31-18-160.ec2.internal', 'Priv
ateIpAddress': '172.31.18.160'}], 'SourceDestCheck': True, 'Status': 'in-use', 'SubnetId': 'subnet-0c1a4c7adc2f937ec', 'VpcId': 'vpc-0a81207dcb2962707', 'InterfaceType': 'in
terface'}, 'RootDeviceName': '/dev/sda1', 'RootDeviceType': 'ebs', 'SecurityGroups': [{'GroupName': 'default', 'GroupId': 'sg-04c3a797e30a4bea5'}], 'SourceDestCheck': True,
'Tags': [{'Key': 'Name', 'Value': 'My Linux Server'}], 'VirtualizationType': 'hvm', 'CpuOptions': {'CoreCount': 1, 'ThreadsPerCore': 1}, 'CapacityReservationSpecification':
{'CapacityReservationPreference': 'open'}, 'HibernationOptions': {'Configured': False}, 'MetadataOptions': {'State': 'applied', 'HttpTokens': 'required', 'HttpPutResponseHop
Limit': 2, 'HttpEndpoint': 'enabled', 'HttpProtocolIpv6': 'disabled', 'InstanceMetadataTags': 'disabled'}, 'EnclaveOptions': {'Enabled': False}, 'BootMode': 'uefi-preferred
', 'PlatformDetails': {'Linux/UNIX': 'UsageOperation': 'RunInstances', 'UsageOperationUpdateTime': datetime.datetime(2026, 9, 28, 15, 42, 46, tzinfo=tzutc()), 'PrivateDnsName
Options': {'HostNameType': 'ip-name', 'EnableResourceNameDnsAAAARecord': True, 'EnableResourceNameDnsAAAARecord': False}, 'MaintenanceOptions': {'AutoRecovery': 'default', 'Cu
rrentInstanceBootMode': 'legacy-bios'}, 'OwnerId': '590269919252', 'ReservationId': 'r-0983374602b2dcce', 'ResponseMetadata': {'RequestId': 'a645e41e-1375-4fb0-99ab-13ac
db941652', 'HTTPStatusCode': 200, 'HTTPHeaders': {'x-amzn-requestid': 'a645e41e-1375-4fb0-99ab-13acdb941652', 'cache-control': 'no-cache, no-store', 'strict-transport-securi
ty': 'max-age=31536000; includeSubdomains', 'vary': 'accept-encoding', 'content-type': 'text/xml; charset=UTF-8', 'transfer-encoding': 'chunked', 'date': 'Tue, 29 Sep 2026 12
:26:39 GMT', 'server': 'AmazonEC2'}, 'RetryAttempts': 0}]
eee.W.6848988@runweb251809:~$

```

Figure: Running the Python3 file

Does the Python3 program show your instances? [Yes/No]

Note, Boto3 has deprecated since Python 3.7, so just force Python to ignore any warnings with (assuming you have named the file 1.py):

```
python3 -W ignore 1.py
```

Now we will stop one of our instances. For this, get an instance ID, and add it to the following file:

```
nano 2.py
```

And paste

```

import boto3
ec2 = boto3.client('ec2', region_name='us-east-1')
ec2.stop_instances(InstanceIds=["i-07b0512e24xxxxxx"]) # where x's are
replaced by your instance's ID

```

Now run the Python file, and prove that it has stopped your instance.

```
python3 -W ignore 2.py
```

Does the Python3 program stop your instance? [Yes/No]

Now we will restart one of our instances. For this, get an instance ID, and add it to the following file:

```
nano 3.py
```

And paste

```
import boto3
ec2 = boto3.client('ec2', region_name='us-east-1')
ec2.start_instances(InstanceIds=["i-07b0512e24xxxxxx"]) # where x's are
replaced by your instance's ID
```

Now run the Python file, and prove that it has started your instance.

```
python3 -W ignore 3.py
```

Does the Python3 program start your instance? [Yes/No]

Finally, write a Python3 program which will start both of your instances, and another one to stop them both.

```
nano do it yourself ;-)
```

Do your Python3 programs work? [Yes/No]

You can also use the AWS prompt. Now try to start and stop your instances with:

```
aws ec2 stop-instances --instance-ids i-07b0512e24xxxxxx --no-cli-pager #
remember to change the xxx's for your instance
```

and

```
aws ec2 start-instances --instance-ids i-07b0512e24xxxxxx --no-cli-pager #
remember to change the xxx's for your instance
```

Do these command line programs work? [Yes/No]

Now we will create a keypair with Python, and then create a new Linux instance. First create the keypair with the Python file of:

```
nano 4.py
import boto3
ec2 = boto3.client('ec2', region_name='us-east-1')
key_pair = ec2.create_key_pair(KeyName='mykeypair2')
MyKeyPair = key_pair["KeyMaterial"]
outfile = open('mykeypair2.pem', 'w')

print(MyKeyPair)
outfile.write(MyKeyPair)
outfile.close()
```

Save, Exit, Run, Check

```
python3 -W ignore 4.py
cat mykeypair2.pem
```

What is the name of your key pair? Can you find it in your AWS Management console? [Yes/No]

```

eee_W_6848988@runweb251800:~$ python3 -W ignore 4.py
-----BEGIN RSA PRIVATE KEY-----
MIIEpQIBAAKCAQEA2PIDi08WrzLoavq09cLHmMgj+GNkvmE0o7Fo5NBXnqHLS9Fh
XTNwKrLCryM65V4Z+u6tSXP14nRVAG3UwqS/KgLEP9AHofyyQpQyJf167FgYJCH
/f08s61Nlp6ywfPjvaoGN6bgcPvplBxL0bzkXwUspwH5a3f2ejeIW5eGfn15+IJb
CzJ70L7Ip4JcZrzkx9ThRIERcw1cSVVbNwCnyPzVP/51cqNc46+NkmabevsYy5/f
rCjkdhdSL7Q/kCqBjXmPeaAt8CwBklfLYHxE+XUGfnCtbvvjSLEmaQwmN7RoqHD
Ea/y15xFcte5+ZnYr1vf10tWTI1tG8L4GWNMbQIDAQABoIBAQC20is/9Vx/k2po
megw2wifS9XzCUIRLCOI2jd6o4pFxEV/dw0Q0UeFmKeY0K4bPSAEYgYiWkom3yZrc
Toi3pxUqRu7DpTQtSRH9Tr3n6L3dWwPUHmjnlrSzcVntD6alhgBRSTn0DOMjoe9
kjsZL0na6tTD/80LQvv1UvolVSKn70V+JzGsHUZ2NnwSUEYgKmqw+1L+KH4J3
uahna4dzT8vHzTbfsoX5BuMmx5IVaD9o+7w7bPMr0mG2Eaxl+1B/GGC6KMnX+Ygd
ZWFMbj3JcoyVzjjv0FzMAEvDMg8+94w01pgDx6IJzsQui13SY8fy63ry9dqaocV3
2NrzkjzAoG8BAPPtPKBYGidyYZBj2fiL/ZkB0SzVjLp1AbUaFrnF+2HQIfSFJs5
eJvRNCfeywFIQCwMSk0tsGe0/p50B6WtbhIWxwPrW5RcYL2g1z86BVWUN2Sgn/8D
nF+/m4mabtP6He2XXH2m0LWwEn+48c0XK+uH5tvmvd8FVr2EsZ/4pHJLAoGBAOPG
zJgM7ZGz2UwoprST6EL1y1v+14W2PP0JakJbVvZJCKnvIj6mL0Sy4ea0d9H0c07f
HoY6uZ46lWnh0IeQkcecnUpqH1mjSXBtInri/bDrGEdI2Pv8vIYweMXvELZRwWE
DUGlqTZDxBGefYywhWcm/Wio09XJRa89wfG4K8knAoGBA05A5cG3S800Q0iMMXwa
OjGyBFIRp24yrHTL/PiujEQNYJIXRwlyvBCNY7Z97RvBR0uFyt+dDh4YbyDocMWE
1WSZmz1KN7+eE03bB9mBcYrxPLt4dT70IU4yZeKs5BtabnomiDQtQMzPY0nLafX
N3p8HqMKsaNaFfgMzLYq1HAoGBA0Mk3eShh+Yz7u+W3L/X29f1mrFRGwYs0DE
wXhFoC2Z+9A70Qc+UJODR7JVk3msTRVBnCFYXmcPFuFmUT/c5EW6nhKCGdu4Y5z
YYPK/SHREJcqQaP+nnq0o8rkPc2tKmBu1CzWcg1sdCwryL207F9dB0BwtJzZT6A0Q
zjbKDCgJAoGAZ68EjVMfDwD2iH9t6PfnxJhaD15gSjdGpSFZTiZa0nk091yqc46F
/H+QhE+hyG6lpSrP7Po0De3j0UsBfhaPjX17oRe9kHe/5wYTurVia2TEVeXzfW19
+0072DCA+6+0p+A/1m1tC4blxqJAt6Yjj4Rh808q5cvmvE0eqB56mGI=
-----END RSA PRIVATE KEY-----
eee_W_6848988@runweb251800:~$

```

Figure: Key Pair

Now we will create a Linux instance. Take a note of the AMI (not the instance ID!) for your Linux instance (see screenshot below)

Instance summary for i-09c47b8513c7181a8 (My Linux Server) [Info](#)

Updated 2 minutes ago

Instance ID i-09c47b8513c7181a8	Public IPv4 address 34.233.134.52 open address
IPv6 address -	Instance state Running
Hostname type IP name: ip-172-31-18-160.ec2.internal	Private IP DNS name (IPv4 only) ip-172-31-18-160.ec2.internal
Answer private resource DNS name IPv4 (A)	Instance type t2.micro
Auto-assigned IP address 34.233.134.52 [Public IP]	VPC ID vpc-0a81207dcb2962707
IAM role -	Subnet ID subnet-0c1a4c7adc2f937ec
IMDSv2 Required	Instance ARN arn:aws:ec2:us-east-1:590269919252:instance/i-09c47b8513c7181a8
Operator -	

[Details](#)
[Status and alarms](#)
[Monitoring](#)
[Security](#)
[Networking](#)
[Storage](#)
[Tags](#)

AMI ID copied

[ami-0b6d9d3d33ba97d99](#)

AMI name
[ubuntu/images/hvm-ssd-gp3/ubuntu-resolute-26.04-amd64-server-202604](#)

Stop protection
 Disabled

Monitoring
 disabled

Allowed image
 -

Launch time
[Tue Sep 29 2026 12:32:04 GMT+0100 \(British Summer Time\)](#) (about 1 hour)

Figure: Find the AMI

Now create

```
nano create.py
```

and save the file:

```
import boto3
ec2 = boto3.resource('ec2', region_name='us-east-1')

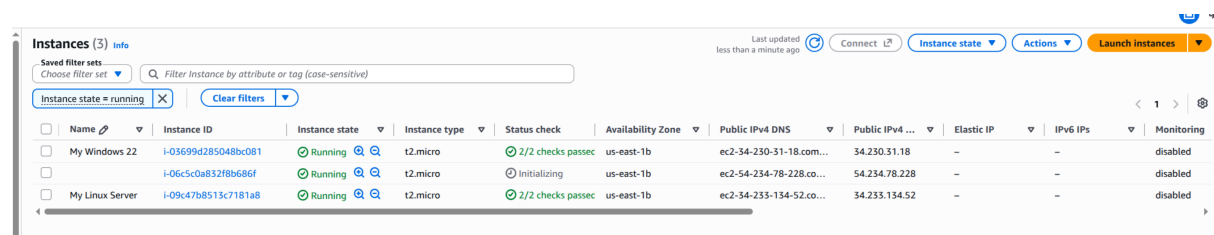
# create a new EC2 instance
instances = ec2.create_instances(
    ImageId='ami-xxxxxxxxxxxxxxxx', # replace with the AMI ID of your
    Ubuntu instance
    MinCount=1,
    MaxCount=1,
    InstanceType='t2.micro',
    KeyName='mykeypair2'
)

print(instances[0].id)
```

Finally run the script.

```
python3 -W ignore create.py
```

Check if it has created it, as in the screenshot below



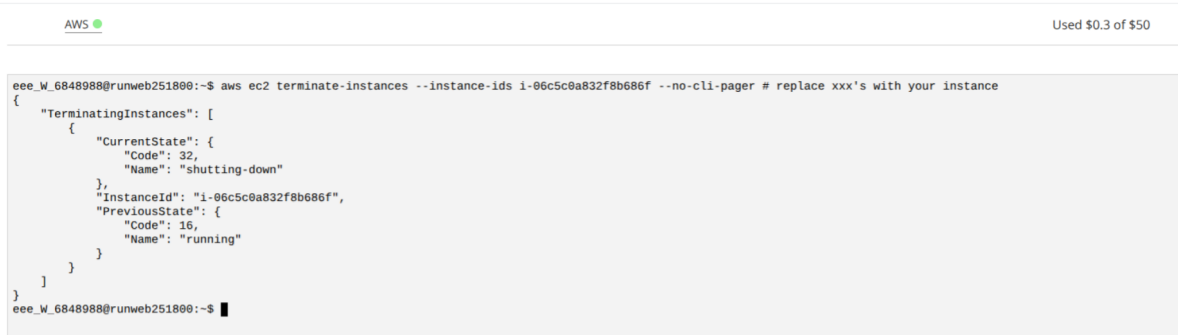
Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS	Public IPv4 ...	Elastic IP	IPv6 IPs	Monitoring
My Windows 22	i-03699d285048bc081	Running	t2.micro	2/2 checks passed	us-east-1b	ec2-34-230-31-18.com...	34.230.31.18	-	-	disabled
My Linux Server	i-06c5c0a832f8b686f	Running	t2.micro	2/2 checks passed	us-east-1b	ec2-54-234-78-228.co...	54.234.78.228	-	-	disabled
	i-09c47b8513c7181a8	Running	t2.micro	2/2 checks passed	us-east-1b	ec2-34-233-134-52.co...	34.233.134.52	-	-	disabled

Figure: New instance created (you may need to refresh the page)

Has it created the instance? [Yes/No]

Now terminate it.

```
aws ec2 terminate-instances --instance-ids i-xxxxxxxxxxxxxxxx --no-cli-
pager # replace xxx's with your instance
```



```
AWS Used $0.3 of $50

eee_w_6848988@runweb251800:~$ aws ec2 terminate-instances --instance-ids i-06c5c0a832f8b686f --no-cli-pager # replace xxx's with your instance
{
  "TerminatingInstances": [
    {
      "CurrentState": {
        "Code": 32,
        "Name": "shutting-down"
      },
      "InstanceId": "i-06c5c0a832f8b686f",
      "PreviousState": {
        "Code": 16,
        "Name": "running"
      }
    }
  ]
}
```

Figure: Terminating your instance

Has it been terminated? [Yes/No]

NOW TERMINATE YOUR NEWLY CREATED INSTANCE (and any others you have created with Python)!

At the end of the lab, you should only have two instances. Please either terminate these, or stop them.

Appendix A

If working on your own Windows Machine, installing WSL is a convenient option. It installs Linux within your Windows operating system and gives you a Linux CLI:

Open PowerShell as Administrator Run

```
wsl --install
```

And restart your PC as prompted. And now you should have Ubuntu living inside your Windows! Convenient. Don't forget the always important updating:

```
sudo apt update && sudo apt upgrade -y
```

To use WSL for the SSH step, the PEM file has to live inside the Linux filesystem, not on /mnt/c/.... Permissions on Windows drives don't really apply, so chmod 400 appears to work but SSH still rejects the key as "unprotected". The fix:

```
cp /mnt/c/Users/<name>/Downloads/mykey.pem ~/ # Copies the key from the
Windows Downloads folder into the Linux home directory, where permissions
actually work
chmod 400 ~/mykey.pem
ssh -i ~/mykey.pem ubuntu@<instance-ip>
```

But if you are using the University's machine, then follow the next steps.

Note, if you are using a Microsoft Windows system, you will have to use icacls through the Command Terminal to make the file read-only. **The following commands must be ran from the windows command prompt.** Once you have opened the command prompt window, please ensure you are in the same directory that the PEM file is stored in. You can run the `dir` command to display the contents of the current directory.

If you are using a domain (such as with NAPIER-MAIL, you would define your domain name with your matriculation number, such as:

```
> icacls.exe mykey.pem /reset
> icacls.exe mykey.pem /grant:r NAPIER-MAIL\xxxxxxx:(r)
> icacls.exe mykey.pem /inheritance:r
```

If you are using your own Windows device, you will first need to run the `whoami` command to determine your domain and username:

```
> whoami
```

Which will display you user's domain and username. For example the output of the `whoami` command could look like:

```
test-domain\test-user
```

Using this information, you will alter the icacls commands to be specific to your user details. For example:

```
> icacls.exe mykey.pem /reset  
> icacls.exe mykey.pem /grant:r test-domain\test-user:(r)  
> icacls.exe mykey.pem /inheritance:r
```

You can copy the PEM file from the JKCC machines to a USB or OneDrive folder so you can use the same PEM file at home, but the permissions must be updated whenever you move it to another machine. It is also important to store the PEM file on the device itself when calling it in the SSH command, rather than keeping it stored in OneDrive.